CONCETTI BASE — Il "dizionario di sopravvivenza"

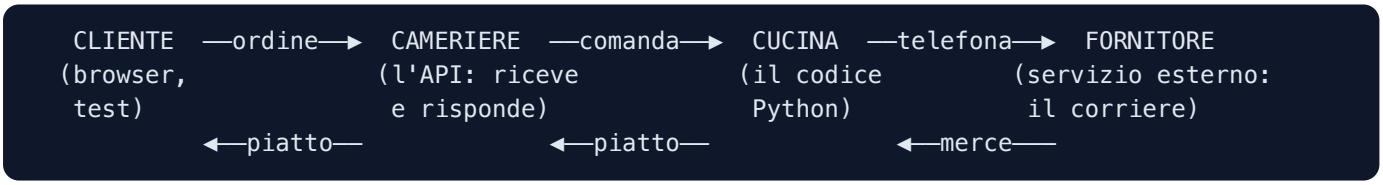
Per chi è questo file? Per tutti. Se durante un lab leggete una parola e non siete **sicuri al 100%** di cosa significhi, venite qui. Non è una vergogna: è esattamente quello che fanno gli sviluppatori senior quando cambiano tecnologia.
Ogni voce ha tre parti:

- **In una frase:** la definizione minima.
- **Come al ristorante:** un'analogia per fissare l'idea.
- **Provalo:** un comando o un gesto per vederlo con i vostri occhi.

Indice

- Parte 1: Reti e web (Giorno 6)
- Parte 2: Python e test (ripasso del semestre 1, usato ogni giorno)
- Parte 3: Servizi esterni e browser (Giorno 7-8)
- Parte 4: Manutenzione del codice (Giorno 9)
- Parte 5: Rilascio in produzione (Giorno 9)

L'analogia che useremo sempre: il ristorante



- Il **cliente** non entra mai in cucina: parla solo con il cameriere, con un **menù** preciso (il contratto).
- Se il cliente ordina un piatto che non esiste, il cameriere dice "non ce l'abbiamo" (**errore 4xx, colpa del cliente**).
- Se la cucina prende fuoco, il cameriere dice "scusi, problema nostro" (**errore 5xx, colpa del server**).
- Se il fornitore non consegna il pesce, la cucina non deve bloccarsi per sempre ad aspettarlo (**timeout**).

Parte 1 — Reti e web (Giorno 6)

Client e server

- **In una frase:** il **client** è il programma che **chiede**; il **server** è il programma che **risponde**.

- **Come al ristorante:** il client è il cliente, il server è il cameriere che porta le risposte.
- **Attenzione:** "server" è un **ruolo**, non per forza un computer potente. Quando lanciate `python app.py`, il vostro portatile **diventa** un server. Il browser e i vostri test sono i client.
- **Provalo:** con `python app.py` acceso, aprite http://localhost:8000. Il browser (client) ha chiesto e il vostro programma (server) ha risposto. Spegnete `app.py` con CTRL+C e ricaricate la pagina: senza server, il client non riceve niente.

Processo

- **In una frase:** un programma **in esecuzione**. Lo stesso file `app.py` lanciato due volte crea due processi.
- **Come al ristorante:** la ricetta è il file; il cuoco che la sta cucinando adesso è il processo.
- **Provalo:** un terminale = un processo che resta acceso finché non premete CTRL+C. Per questo nella Bottega servono **più terminali**: uno per ogni processo.

Terminale (e perché ne servono tre)

- **In una frase:** la finestra dove scrivete i comandi. Un terminale occupato da un server **non** può fare altro finché il server è acceso.
- **Nella Bottega:** terminale 1 = corriere, terminale 2 = app, terminale 3 = i vostri comandi (`pytest`). In **ognuno** va attivato il venv.

localhost e indirizzo IP

- **In una frase:** ogni computer in rete ha un **indirizzo** (IP). `localhost` (o `127.0.0.1`) è un indirizzo speciale che significa "**questo** computer".
- **Come al ristorante:** è come dire "il ristorante in cui mi trovo già", senza dare l'indirizzo della via.
- **Provalo:** http://localhost:8000 funziona solo sul vostro PC. Se il vicino di banco apre lo stesso indirizzo, vede il **suo** computer, non il vostro.

Porta

- **In una frase:** un numero (da 1 a 65535) che identifica **quale programma** risponde, sullo stesso computer.
- **Come al ristorante:** l'indirizzo è la via del centro commerciale, la porta è il **numero del negozio**.
- **Nella Bottega:** porta **8000** = La Bottega, porta **8001** = il corriere. Stesso computer, due negozi diversi.
- **Errore tipico:** `Address already in use` vuol dire "c'è già qualcuno in quel negozio": avete lanciato il programma due volte.

URL

- **In una frase:** l'indirizzo completo di una risorsa. Si legge così:

```
  http://localhost:8000/api/prodotti/2?formato=breve
  └─┬─┘  └──────┬─────┘ └┬┘└──────┬──────┘ └──────┬──────┘
protocollo  computer porta   percorso     parametri
                           (QUALE cosa)  (dettagli opzionali)
```

HTTP

- **In una frase:** la **lingua** con cui client e server si parlano sul web. Ogni scambio è sempre una coppia **richiesta → risposta**.
- **Come al ristorante:** è il modo standard di ordinare: "Vorrei... (piatto) per... (tavolo)". Tutti i camerieri del mondo lo capiscono.

Richiesta e risposta HTTP

```
RICHIESTA (il client manda)                RISPOSTA (il server restituisce)
─────────────────────────────              ──────────────────────────────────
METODO   POST                              STATUS   201 Created
URL      /api/carrello                     HEADER   Content-Type: application/json
HEADER   Authorization: Bearer abc123
BODY     {"prodotto_id": 2,                BODY     {"prodotto_id": 2,
          "quantita": 1}                             "quantita": 1}
```

Metodi HTTP (i "verbi")

Metodo	Significato	Al ristorante
`GET`	**leggi** qualcosa, senza modificarlo	"Mi fa vedere il menù?"
`POST`	**crea** o invia qualcosa	"Vorrei ordinare questo"
`PUT` / `PATCH`	**modifica** qualcosa	"Cambi il contorno, per favore"
`DELETE`	**cancella** qualcosa	"Annulli l'ordine"

Status code

- **In una frase:** il numero di 3 cifre con cui il server dice **com'è andata**. È l'equivalente dell'**exit code** del semestre 1.
- **La regola delle centinaia:** `2xx` = tutto ok · `4xx` = colpa del **client** · `5xx` = colpa del **server**.
- Quelli che userete: `200` ok, `201` creato, `400` dati sbagliati, `401` non so chi sei, `403` so chi sei ma non puoi, `404` non esiste, `409` conflitto (es. stock finito), `500` il server è esploso, `503` un servizio da cui dipendo non risponde.

Header

- **In una frase:** informazioni **sulla** richiesta o sulla risposta (non il contenuto vero e proprio).
- **Come al ristorante:** il biglietto pinzato sulla comanda: "tavolo 7, allergico alle noci".
- Esempi: `Content-Type: application/json` ("ti mando JSON"), `Authorization: Bearer abc123` ("ecco il mio pass").

Body

- **In una frase:** il **contenuto** della richiesta o della risposta (i dati veri e propri). Un `GET` di solito non ha body; un `POST` sì.

JSON

- **In una frase:** un formato di **testo** per scrivere dati strutturati. Somiglia moltissimo a un dizionario Python.

```
# Dizionario Python                    # Testo JSON (quello che viaggia in rete)
{"nome": "Mouse", "prezzo": 34.5,      {"nome": "Mouse", "prezzo": 34.5,
 "disponibile": True, "note": None}     "disponibile": true, "note": null}
```

- **Attenzione:** JSON è **testo**. `risposta.json()` lo trasforma in un dizionario Python; con `json=` nei test `requests` fa il contrario per voi.
- **Provalo:** aprite http://localhost:8000/api/prodotti nel browser: quello che vedete è JSON.

API ed endpoint

- **In una frase:** l'**API** è l'insieme delle "porte d'ingresso" che un programma offre ad altri programmi. Ogni porta d'ingresso (metodo + percorso, es. `GET /api/prodotti`) è un **endpoint**.
- **Come al ristorante:** l'API è il **menù**; ogni piatto del menù è un endpoint.
- **REST** è uno **stile** per progettare API: risorse con nomi chiari (`/prodotti`, `/carrello`) e verbi HTTP per le azioni.

Contratto (specifica)

- **In una frase:** il documento che dice **esattamente** cosa fa ogni endpoint: cosa riceve, cosa restituisce, quali errori. Per noi è `bottega/README.md`.
- **Regola del corso:** se codice e contratto non sono d'accordo, **ha ragione il contratto** e il codice ha un bug.

Frontend e backend

- **Frontend:** quello che gira **nel browser** e che l'utente vede (HTML, CSS, JavaScript): `bottega/static/`.
- **Backend:** quello che gira **sul server** e che l'utente non vede (Python): `bottega/app.py`.
- Il frontend parla con il backend **tramite l'API**, esattamente come fanno i vostri test.

Autenticazione e token

- **In una frase:** dimostrare **chi sei**. Con il login il server vi dà un **token**: una stringa casuale da allegare a ogni richiesta successiva, nell'header `Authorization: Bearer <token>`.
- **Come al ristorante:** il braccialetto del villaggio turistico. Lo mostri una volta all'ingresso, poi ogni bar ti serve senza chiederti i documenti.

Parte 2 — Python e test (ripasso del semestre 1, usato ogni giorno)

Virtual environment (venv)

- **In una frase:** una cartella che contiene un Python **isolato** con le sue librerie, solo per questo progetto.
- **Il segnale:** se nel terminale vedete `(venv)` all'inizio della riga, è attivo. Ogni terminale nuovo va attivato di nuovo.

Libreria (pacchetto) e `pip`

- **In una frase:** codice scritto da altri, pronto da usare. `pip install` la scarica, `requirements.txt` è la lista della spesa.
- Le nostre: `flask` (per fare il server), `requests` (per fare il client), `pytest` (per i test).

Funzione, parametro, valore di ritorno

```
def costo_spedizione(cap: str, peso: float) -> float:   # cap e peso sono PARAMETRI
    ...
    return 4.9                                          # 4.9 è il VALORE DI RITORNO
```

Dizionario

- **In una frase:** una collezione di coppie **chiave → valore**: `prodotto["prezzo"]` legge il valore della chiave `"prezzo"`.
- `KeyError: 'quantita'` vuol dire: "avete chiesto una chiave che nel dizionario non c'è".

Eccezione

- **In una frase:** il "freno di emergenza" di Python: interrompe il programma quando qualcosa va storto. `raise` lo tira, `try/except` lo gestisce, `pytest.raises` verifica che venga tirato.
- **Come si legge un traceback:** dal **basso verso l'alto**. L'ultima riga dice **cosa** è successo, le righe sopra dicono **dove**.

Import e modulo

- **In una frase:** ogni file `.py` è un **modulo**. `import spedizioni` vi dà accesso al codice di `spedizioni.py`.

Pattern AAA

- **Arrange** (prepara) → **Act** (esegui) → **Assert** (verifica). Ogni test, sempre, in quest'ordine.

Mock

- **In una frase:** un oggetto **finto** che prende il posto di uno vero durante il test, e a cui diciamo noi cosa rispondere (o come esplodere).
- **Come al ristorante:** un attore che fa la parte del fornitore durante l'addestramento dei cuochi. Così si può provare "il fornitore non arriva" senza aspettare che succeda davvero.

Fixture (pytest)

- **In una frase:** una funzione che **prepara qualcosa** per i test (un login, uno stato pulito) e lo consegna a chi lo chiede **per nome**.
- **Come al ristorante:** la **mise en place**: prima del servizio si prepara tutto quello che serve, una volta sola, e ogni cuoco lo trova pronto.

Test isolato e test flaky

- **In una frase:** un test è **isolato** se passa o fallisce sempre allo stesso modo, in qualunque ordine e quante volte lo lanciate. Se il risultato cambia da un'esecuzione all'altra, il test è **flaky** (ballerino).
- **Come al ristorante:** ogni cuoco trova la postazione pulita. Se trova gli avanzi del collega, il suo piatto viene diverso ogni volta.
- **La cura che usiamo:** la fixture `stato_pulito` con `autouse=True`, che prima di **ogni** test chiama `POST /api/reset`.

`parametrize`

- **In una frase:** scrivere un test **una volta sola** e farlo girare su tanti casi (una tabella di input e risultati attesi). pytest mostra ogni caso come un test separato.

Black box e white box

- **White box:** guardo **dentro** il codice (import, mock, if). **Black box:** guardo solo **da fuori** (input → output), come farebbe un utente.

Parte 3 — Servizi esterni e browser (Giorno 7-8)

Servizio esterno (dipendenza)

- **In una frase:** un programma di **un'altra azienda** da cui il nostro dipende. Non possiamo modificarlo, possiamo solo chiamarlo. Per noi è il corriere SpediVeloce.
- **Come al ristorante:** il fornitore del pesce. Se non consegna, la cucina deve sapere cosa fare.

Timeout

- **In una frase:** il **tempo massimo** che siamo disposti ad aspettare una risposta. Scaduto quello, si rinuncia e si gestisce l'errore.

- **Come al ristorante:** "se il fornitore non arriva entro le 11, togliamo il pesce dal menù di oggi". Senza questa regola, la cucina resta ferma tutto il giorno.

Eccezione di dominio

- **In una frase:** un'eccezione **nostra**, con un nome che parla la lingua del progetto (`ServizioSpedizioniNonDisponibile`). Il resto del codice non deve sapere se il corriere è caduto per un timeout o per un errore di rete: gli basta sapere "il servizio non c'è".
- **Come al ristorante:** il cameriere non spiega al cliente che il furgone del fornitore ha bucato. Dice "oggi il pesce non c'è".

Guasto simulato (`side_effect`)

- **In una frase:** con `side_effect` diciamo al mock di **esplodere** invece di rispondere (es. `mock_get.side_effect = requests.Timeout`). Così proviamo in un millisecondo un guasto che nella realtà capita una volta al mese.

Sistema distribuito

- **In una frase:** un sistema fatto di **più processi** che collaborano **attraverso la rete**. La Bottega + il corriere sono già un sistema distribuito, in piccolo.

HTML, CSS, JavaScript

	Cosa fa	Analogia (casa)
HTML	la **struttura**: titoli, bottoni, campi	i muri e le stanze
CSS	l'**aspetto**: colori, spazi, font	l'arredamento e la pittura
JavaScript	il **comportamento**: cosa succede al click	l'impianto elettrico

DOM

- **In una frase:** la rappresentazione "ad albero" della pagina che il browser costruisce leggendo l'HTML. Playwright clicca e legge **elementi del DOM**.
- **Provalo:** tasto destro su un elemento della pagina → **Ispeziona**.

Elemento, tag, attributo

```
<button type="submit" id="conferma">Conferma ordine</button>
 └tag┘ └attributo┘ └attributo┘ └─── testo ───┘
```

Selettore / locator

- **In una frase:** il modo per dire a Playwright **quale** elemento cercare. Il migliore è "come lo cercherebbe una persona": `get_by_role("button", name="Conferma ordine")`.

Headless

- **In una frase:** un browser **senza finestra**: funziona davvero, ma non lo vedete. È così che i test girano in CI.

Test end-to-end (E2E)

- **In una frase:** un test che usa l'applicazione **come un utente vero**, dal browser fino al database, passando per tutti i pezzi in mezzo.
- **La piramide dei test:** tanti test unitari (veloci, con il mock), un po' di test sulle API, **pochi** test E2E (lenti e delicati, ma gli unici che vedono quello che vede il cliente).

Attesa automatica (`expect`)

- **In una frase:** `expect(...)` di Playwright **riprova da solo** finché la condizione è vera o finisce il tempo massimo. Per questo nei test non si usa mai `time.sleep()` : o aspetta troppo, o troppo poco.

Accessibilità

- **In una frase:** fare in modo che il sito sia usabile **da tutti**, anche da chi non vede, usa solo la tastiera o ha difficoltà visive. Il **lettore di schermo** è il programma che legge la pagina ad alta voce.
- **axe:** uno scanner automatico che controlla la pagina contro le regole WCAG (contrasto, etichette, lingua) e dice **quali** elementi le violano.

Parte 4 — Manutenzione del codice (Giorno 9)

Codice legacy

- **In una frase:** codice che funziona (forse), che nessuno capisce più fino in fondo, e **senza test**.

Refactoring

- **In una frase:** cambiare **come** è scritto il codice senza cambiare **cosa** fa. Come riordinare una stanza: le cose restano le stesse, ma si trovano meglio.

Bug di regressione

- **In una frase:** qualcosa che **prima funzionava** e che una modifica ha rotto. I test esistono soprattutto per accorgersene subito.

Debito tecnico

- **In una frase:** le scorciatoie prese oggi che costeranno lavoro domani, come un prestito con gli interessi.

Parte 5 — Rilascio in produzione (Giorno 9)

Ambiente (sviluppo, test, produzione)

- **Sviluppo:** il vostro PC. **Produzione:** il server vero, usato dai clienti veri. Un errore in sviluppo costa un minuto, in produzione costa soldi e clienti.

Deploy

- **In una frase:** l'operazione di **installare** una nuova versione del software sul server di produzione.

Cloud / PaaS

- **In una frase:** computer di un'altra azienda che si affittano via internet. Un **PaaS** (es. Render) si occupa anche di installare Python e avviare l'app: voi date solo il codice.

Variabile d'ambiente

- **In una frase:** un valore di configurazione passato al programma **dall'esterno**, senza scriverlo nel codice (es. `BOTTEGA_ENV=produzione`). È lo stesso principio del file `.env` del semestre 1.

Segreto (secret)

- **In una frase:** un valore che **non deve finire su Git** (password, chiavi, URL di deploy). Su GitHub si salva in *Settings* → *Secrets*.

Pipeline CI/CD

- **CI** (integrazione continua): a ogni push, un server esegue i test. **CD** (rilascio continuo): se i test passano, il server **installa anche** la nuova versione in produzione.

Smoke test

- **In una frase:** un test **veloce e superficiale** subito dopo il deploy: "il server si accende e risponde?". Il nome viene dall'elettronica: accendi il circuito e guardi se esce fumo.